Learning From Data

Mihir Rao

Contents

1 The Learning Problem **2**
- 1.1 Learning from examples . . . 2
- 1.2 Components of the learning problem . . . 2
- 1.3 The perceptron hypothesis set . . . 3
- 1.4 The perceptron learning algorithm . . . 3
- 1.5 Types of learning . . . 4
- 1.6 A learning puzzle . . . 5

2 Is Learning Feasible? **6**
- 2.1 Probability replaces certainty . . . 6
- 2.2 From marbles to prediction errors . . . 7
- 2.3 Choosing a hypothesis changes the question . . . 8
- 2.4 A guarantee for a finite hypothesis set . . . 9

Lecture 1: The Learning Problem

1.1 Learning from examples

Machine learning infers a rule from observations. It is useful when there is structure to exploit, a satisfactory rule is unknown, and examples are available.

Example: *Movie recommendations.* Predict a viewer's rating by matching a vector of viewer preferences with a vector of movie attributes and summing their contributions. Comedy, action, and particular actors suggest possible factors, but the factors themselves can be learned from observed ratings. The data determine the representation as well as the predictions.

1.2 Components of the learning problem

For credit approval, an input describes an applicant, using attributes such as income and debt, and the output indicates whether the applicant is a good credit risk. The ideal decision rule is unknown; historical records provide examples of its behavior.

Definition 1.1 [Learning setup]. Let $\mathcal{X}$ be the input space and $\mathcal{Y}$ the output space. An unknown target function $f : \mathcal{X} \to \mathcal{Y}$ supplies the labels in the training set

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_N, y_N)\}, \qquad y_n = f(\mathbf{x}_n).$$

A hypothesis set $\mathcal{H}$ contains the candidate functions $h : \mathcal{X} \to \mathcal{Y}$. A learning algorithm $\mathcal{A}$ uses $\mathcal{D}$ to choose a final hypothesis $g \in \mathcal{H}$ that should approximate f on new inputs.

The pair $(\mathcal{H}, \mathcal{A})$ is the learning model. The target and observations describe the problem; the hypothesis set and algorithm are the two components we choose to solve it.

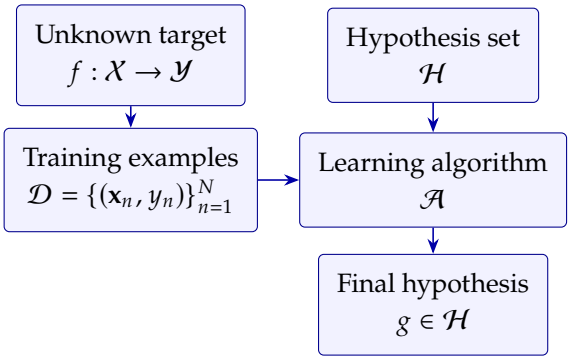


Figure 1: The target supplies training labels. The algorithm sees examples and candidate hypotheses, then returns g; it has no direct access to f.

1.3 The perceptron hypothesis set

For binary credit decisions, take $\mathcal{Y} = \{-1, +1\}$, with $+1$ representing approval. A PERCEPTRON compares a weighted sum of the d input attributes with a threshold τ:

$$h(\mathbf{x}) = \operatorname{sign}\left(\sum_{i=1}^{d} w_i x_i - \tau\right).$$

Absorb the threshold into a bias $w_0 = -\tau$ and append the constant coordinate $x_0 = 1$. From here on, $\mathbf{x} = (1, x_1, \ldots, x_d)^{\mathsf{T}}$ and $\mathbf{w} = (w_0, w_1, \ldots, w_d)^{\mathsf{T}}$, so

$$h(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x}), \qquad \mathcal{H} = \{\mathbf{x} \mapsto \operatorname{sign}(\mathbf{w}^{\mathsf{T}}\mathbf{x}) : \mathbf{w} \in \mathbb{R}^{d+1}\}.$$

In the original attribute space, the decision boundary is $w_0 + \sum_{i=1}^{d} w_i x_i = 0$: a line in two dimensions and a hyperplane in higher dimensions, provided $(w_1, \ldots, w_d) \neq 0$.

Definition: The training data are LINEARLY SEPARABLE if some $\mathbf{w}$ places every example strictly on its correct side: $y_n \mathbf{w}^{\mathsf{T}}\mathbf{x}_n > 0$ for all n.

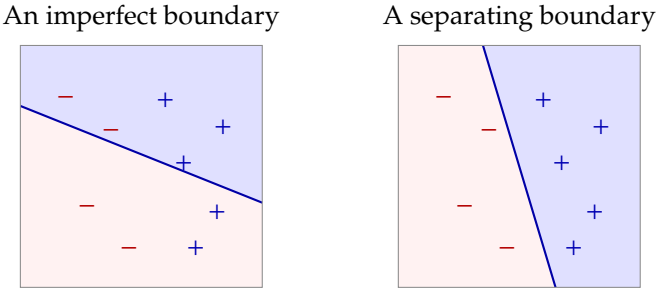


Figure 2: Schematic data with two candidate boundaries. The left boundary makes mistakes; the right one separates all training examples.

1.4 The perceptron learning algorithm

The PERCEPTRON LEARNING ALGORITHM (PLA) repeatedly chooses a misclassified training example $(\mathbf{x}_n, y_n)$ and updates

$$\boxed{\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n.}$$

For the procedure below, a zero score also triggers an update. This avoids assigning a boundary point to either class during training.

Algorithm 1.1: Perceptron learning algorithm

Input: Augmented examples $(\mathbf{x}_n, y_n)$, with $x_{n,0} = 1$ and $y_n \in \{-1, +1\}$
Output: A classifier that separates the training data, if the loop terminates

$\mathbf{w} \leftarrow \mathbf{0}$
while some n satisfies $y_n \mathbf{w}^\mathsf{T} \mathbf{x}_n \le 0$ **do**
    choose such an index n
    $\mathbf{w} \leftarrow \mathbf{w} + y_n \mathbf{x}_n$
return $g(\mathbf{x}) = \operatorname{sign}(\mathbf{w}^\mathsf{T} \mathbf{x})$

For a positive example, add $\mathbf{x}_n$; for a negative one, subtract it. The update moves the chosen example's score toward its correct sign:

$$y_n \mathbf{w}_{\text{new}}^\mathsf{T} \mathbf{x}_n = y_n \mathbf{w}_{\text{old}}^\mathsf{T} \mathbf{x}_n + \|\mathbf{x}_n\|^2.$$

This increases its signed score but need not correct it in one step, and other examples can become misclassified. The augmented update also adjusts the bias: $w_0 \leftarrow w_0 + y_n$.

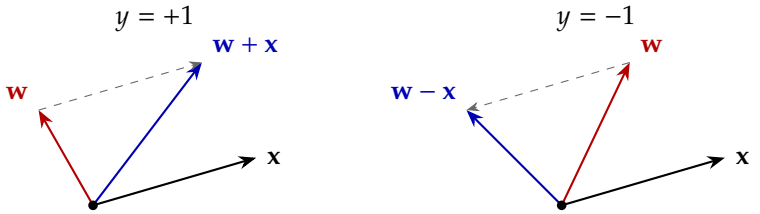


Figure 3: Schematic vector updates for misclassified examples. Dashed arrows complete the addition of $y\mathbf{x}$ to $\mathbf{w}$.

If the procedure stops, all training examples are correctly classified. Without a separating hyperplane, this stopping condition cannot be reached. Agreement on the training set still leaves the question of performance on new inputs.

1.5 Types of learning

The available feedback distinguishes three learning paradigms.

- Supervised learning receives input–label pairs $(\mathbf{x}, y)$. In coin recognition, mass and size are inputs, and the denomination is the label. Labeled examples determine decision regions for future coins.
- Unsupervised learning receives inputs without labels. The same coin measurements can reveal clusters, but their denominations are not supplied.
- Reinforcement learning receives feedback evaluating a chosen output or action, rather than the correct action itself. A backgammon player learns from game outcomes; a win or loss evaluates play without specifying the best move at every position.

1.6 A learning puzzle

Six labeled grids constrain an unknown binary rule. What label should be assigned to the new grid?

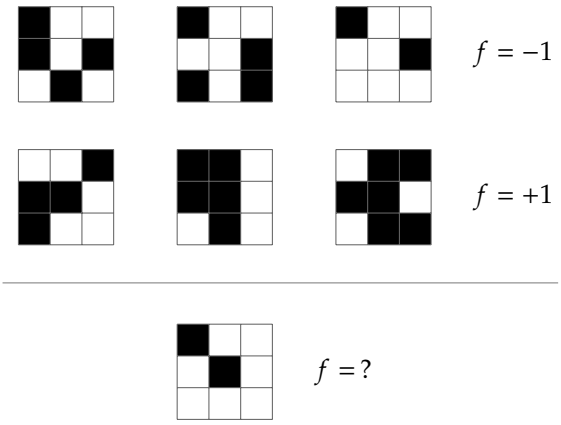


Figure 4: The six examples and unlabeled query in the learning puzzle. Black and white cells are the input attributes.

One consistent rule assigns +1 exactly when the center cell is black. It predicts +1 for the query. Another assigns +1 when the center is black *and* at least three cells are black. It fits the same six examples but predicts −1 for the query, which has only two black cells.

Thus, the examples alone do not uniquely determine the unseen label. Choosing a hypothesis set restricts which rules count as plausible; fitting the observations is only part of the learning problem.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 1: The Learning Problem, Caltech, April 3, 2012. Lecture slides.

Lecture 2: Is Learning Feasible?

Observations cannot determine an arbitrary target function at every unseen input. Two functions can agree on the entire training set and disagree elsewhere. A useful guarantee must instead relate performance on a random sample to performance on the population from which it was drawn.

2.1 Probability replaces certainty

Consider independent draws of red or green marbles, each red with an unknown probability μ. Let $Z_n = 1$ for a red draw and $Z_n = 0$ for a green one. The observed red fraction in N draws is

$$\nu = \frac{1}{N} \sum_{n=1}^{N} Z_n, \qquad \mathbb{E}[Z_n] = \mu.$$

An unrepresentative sample remains possible: a mostly red population can produce a mostly green sample. The question is how *probable* such a discrepancy is.

Theorem 2.1 [Hoeffding's inequality for a binary sample]. *For N independent Bernoulli observations with common mean μ and any $\epsilon > 0$,*

$$\mathbb{P}\big(|\nu - \mu| > \epsilon\big) \leq 2e^{-2N\epsilon^2}.$$

The bound holds for every $\mu \in [0, 1]$, without knowing its value. It decreases exponentially with N for a fixed tolerance ϵ; demanding a smaller tolerance requires a larger sample. A bound above one is valid but uninformative.

This is a probably approximately correct (PAC) statement: the estimate is within ϵ of the truth except on a small fraction of possible samples. Here μ is fixed and ν varies with the sample; the probability concerns the sampling procedure.

2.2 From marbles to prediction errors

Fix a target $f : \mathcal{X} \to \mathcal{Y}$ and a hypothesis $h : \mathcal{X} \to \mathcal{Y}$. Each marble represents an input $\mathbf{x} \in \mathcal{X}$: color it red when $h(\mathbf{x}) \neq f(\mathbf{x})$ and green when the two agree.

Draw $\mathbf{x}_1, \ldots, \mathbf{x}_N$ independently from a distribution P on $\mathcal{X}$, and set $y_n = f(\mathbf{x}_n)$. The distribution need not be known, but it must also describe the new inputs on which performance is measured.

Definition 2.2 [In-sample and out-of-sample error]. For classification, the in-sample error is the fraction of training examples misclassified by h:

$$E_{\text{in}}(h) = \frac{1}{N} \sum_{n=1}^{N} \mathbf{1}\{h(\mathbf{x}_n) \neq y_n\}.$$

The out-of-sample error is its probability of a mistake on a fresh input from P:

$$E_{\text{out}}(h) = \mathbb{P}_{\mathbf{x} \sim P}\big(h(\mathbf{x}) \neq f(\mathbf{x})\big).$$

The indicator $\mathbf{1}\{B\}$ equals one when the statement B holds and zero otherwise. In the marble analogy, $\nu = E_{\text{in}}(h)$ and $\mu = E_{\text{out}}(h)$.

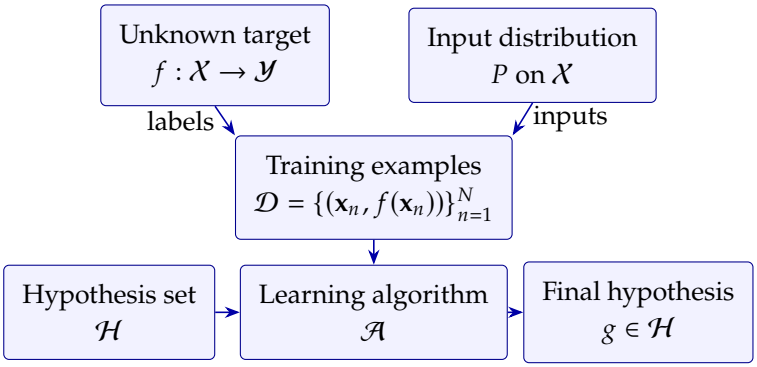


Figure 5: The distribution supplies training inputs and the target supplies their labels. The same P defines out-of-sample performance.

For a hypothesis fixed independently of the training sample, the error indicators are independent Bernoulli observations with common mean $E_{\text{out}}(h)$. Consequently,

$$\mathbb{P}_{\mathcal{D}}(|E_{\text{in}}(h) - E_{\text{out}}(h)| > \epsilon) \leq 2e^{-2N\epsilon^2}.$$

The subscript $\mathcal{D}$ emphasizes that this probability is over the random training set. A close estimate does not itself imply a small error: it may accurately reveal that h performs poorly.

2.3 Choosing a hypothesis changes the question

Learning searches a hypothesis set for a rule with small training error. The returned $g = \mathcal{A}(\mathcal{D})$ therefore depends on the same data used to compute $E_{\text{in}}(g)$. We cannot directly substitute this selected g for the fixed h in the preceding bound.

Example: *Searching for an unusual coin.* For one fair coin flipped ten times,

$$\mathbb{P}(\text{ten heads}) = 2^{-10} \approx 0.098\%.$$

For 1,000 fair coins with all flips independent, the probability that *some* coin produces ten heads is

$$1 - (1 - 2^{-10})^{1000} \approx 62.36\%.$$

Selecting an all-heads coin after inspecting the results does not make that coin biased. An event rare for one fixed choice can be common when there are many opportunities to find it.

For $\mathcal{H} = \{h_1, \ldots, h_M\}$, each hypothesis defines its own red/green coloring of the input space, with population red probability $E_{\text{out}}(h_m)$ and observed red fraction $E_{\text{in}}(h_m)$. The tempting choice is a hypothesis whose sample is mostly green, even if that sample is unusually favorable.

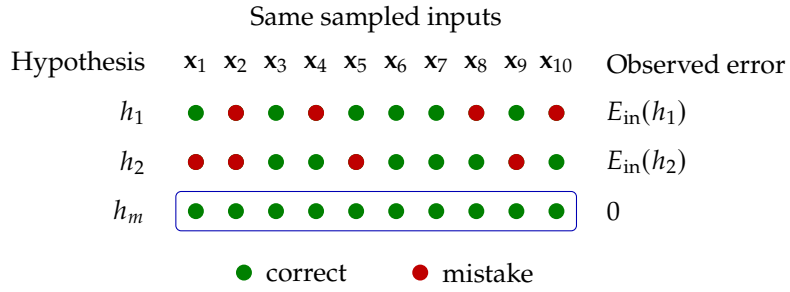


Figure 6: Different hypotheses color the same sampled inputs differently. An all-green row means zero training error, but does not establish zero population error. The colors illustrate the selection issue.

The hypotheses' sample errors need not be independent: they are computed on the same $\mathcal{D}$. Independence of the *training inputs* is what lets us apply Hoeffding to each fixed hypothesis separately.

2.4 A guarantee for a finite hypothesis set

Theorem 2.3 [Finite-hypothesis generalization bound]. *Let $\mathcal{H} = \{h_1, \dots, h_M\}$ be fixed before observing the data. For N independent training inputs from P, labeled by f, and any algorithm returning $g \in \mathcal{H}$,*

$$\mathbb{P}_{\mathcal{D}}\big(|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\big) \le 2Me^{-2N\epsilon^2}.$$

Proof. Write $B_m = \{|E_{\text{in}}(h_m) - E_{\text{out}}(h_m)| > \epsilon\}$. If the selected g has a large discrepancy, then at least one candidate does. Hence

$$\begin{aligned}\mathbb{P}_{\mathcal{D}}\big(|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon\big) &\le \mathbb{P}_{\mathcal{D}}\left(\bigcup_{m=1}^{M} B_m\right) \\ &\le \sum_{m=1}^{M} \mathbb{P}_{\mathcal{D}}(B_m) \\ &\le \sum_{m=1}^{M} 2e^{-2N\epsilon^2} = 2Me^{-2N\epsilon^2}.\end{aligned}$$

The second step is the union bound, which requires no independence among the events B_m. The last step applies Hoeffding to each fixed h_m. □

The argument controls all candidates simultaneously, so it remains valid after selecting one using the data. For a desired failure probability $\delta \in (0, 1)$, solving $2Me^{-2N\epsilon^2} \le \delta$ gives

$$N \ge \frac{\log(2M/\delta)}{2\epsilon^2},$$

where log is the natural logarithm. With this many examples, with probability at least $1 - \delta$,

$$E_{\text{out}}(g) \le E_{\text{in}}(g) + \epsilon.$$

Learning therefore needs both a small training error and a reliable link between training and population errors. A larger hypothesis set offers more candidate rules, but increases the factor M in the guarantee; a larger sample can offset that cost. This argument establishes the link for a finite set, while leaving the algorithm responsible for finding a rule with small training error.

Source. Yaser S. Abu-Mostafa, *Learning From Data*, Lecture 2: Is Learning Feasible?, Caltech, April 5, 2012. Lecture slides.